Real Estate Property Estimation Recommendation and Market Analytics

योगः कर्मसु कौशलम्
IN PURSUIT OF PERFECTION

submitted in partial satisfaction of the requirements
for the degree of
Bachelor of Computer Applications
In Computer Science

ज्योतिर्वृणीत तमसो विजानन

Affiliated to
Guru Gobind Singh Indraprastha University, Delhi

Submitted to:
Assistant Professor
MS. Shalini Bhartiya

Submitted by:
Abhay
08117702023

ACKNOWLEDGEMENTS

In the “Acknowledgements” page, the writer recognizes his indebtedness for guidance and assistance of the thesis adviser and other members of the faculty. Courtesy demands that he also recognize specific contributions by other persons or institutions such as libraries and research foundations. Acknowledgements should be expressed simply, tastefully, and tactfully.

SELF CERTIFICATE

This is to certify that the dissertation/project report entitled “Machine Learning Based Property Estimation Recommendation and Market Analytics” is done by me and an authentic work carried out for the partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications under the guidance of Professor MS. Shalini Bhartiya. The matter embodied in this project work has not been submitted earlier for the award of any degree or diploma to the best of my knowledge and belief.

Signature of the student: Abhay
Name of the Student: Abhay
Roll No: 08117702023

Synopsis of the Project

Title:
Real Estate Property Analytics

Objective:
The main objective of this project is to build an AI-powered real estate analytics platform that helps users predict property prices, explore market trends, and receive personalized property recommendations based on data-driven insights.
The system aims to make the property search process more efficient, transparent, and intelligent through the use of machine learning, data visualization, and similarity-based recommendation algorithms.

Problem Definition:
In the rapidly evolving real estate market, property buyers and investors often face challenges such as price uncertainty, data inconsistency, and lack of personalized recommendations. Existing platforms mainly focus on listing properties without offering analytical insights or predictive intelligence.
Hence, there is a strong need for a system that can analyze historical data, forecast property prices, visualize market patterns, and recommend best-fit properties based on user preferences.

Proposed Solution:
The proposed **Real Estate Analytics System** addresses these challenges by integrating multiple AI-based modules into a unified platform:

1. **Price Prediction Module:** Predicts the estimated market value of a property using machine learning regression algorithms such as Random Forest and XGBoost.
2. **Analytics & Visualization Module:** Generates interactive dashboards to analyze property trends, price distributions, and location-based insights using Plotly and Matplotlib.
3. **Recommendation System:** Suggests the most relevant properties based on user inputs, using a cosine similarity approach with adjustable weights for facilities, pricing, and location.

Section 1: Problem Statement

- Real Estate Analytics is a data-driven platform designed to analyze and visualize real estate trends in Gurugram, helping investors, buyers, and realtors make informed decisions.
- The project focuses on data cleaning, and advanced analytics to extract valuable insights from real estate listings, addressing challenges such as price fluctuations, demand forecasting, property recommendation, investment potential, and property valuation accuracy.
- Unlike traditional property search websites, this project emphasizes data-driven decision-making, using Exploratory Data Analysis (EDA), feature selection, and machine learning to uncover hidden patterns and relationships in real estate data.
- Users can explore interactive dashboards to compare property prices based on location, amenities, property type, and market trends while gaining insights into neighborhood development, buyer demand, and seasonal price variations.
- The platform incorporates predictive analytics to estimate future price trends and rental yields, helping users identify lucrative investment opportunities and avoid potential risks.
- The project integrates Bayesian Optimization using Optuna to fine-tune machine learning models, enhancing the accuracy of price prediction, demand analysis, and investment feasibility forecasting.
- A spatial analysis module allows users to visualize real estate hotspots and compare areas based on factors like connectivity, infrastructure growth, and historical price trends.
- This analytical approach benefits buyers by providing transparency in real estate pricing, helping them make smarter and more profitable investment choices based on objective data rather than speculation.
- The system supports multi-criteria filtering, enabling users to analyze properties based on budget, proximity to key locations (schools, metro stations, hospitals), property size, and builder reputation.
- The platform can be expanded to include real-time data updates, AI-powered property recommendations, integration with rental market trends, and predictive models for real estate appreciation.
- Future enhancements could include automated sentiment analysis from user reviews, an AI-based chatbot for real estate queries, and blockchain-based smart contracts for secure transactions.
- The project is designed to be scalable, adaptable, and future-proof, ensuring that it remains relevant in an ever-evolving real estate market.

Software Requirement Specification (SRS)

1. Introduction

1.1 Purpose

Real Estate Analytics is a data-driven project designed to provide valuable insights into the real estate market in Gurugram. It leverages data science and machine learning techniques to analyze property listings, price trends, and investment opportunities. The primary goal of this project is to help homebuyers, investors, and real estate professionals make informed decisions by offering interactive visualizations, price predictions, and locality-based insights.

A key feature of this project is the ability to scrape real estate data, clean it, and perform exploratory data analysis (EDA) to uncover patterns in pricing, demand, and location-based trends. Additionally, Bayesian Optimization using Optuna has been integrated to fine-tune machine learning models, improving prediction accuracy.

This project aims to bridge the gap between raw real estate data and actionable insights, making it easier for users to understand market dynamics and identify profitable investment opportunities. Future enhancements may include real-time data updates, AI-driven recommendations, and integration with rental market trends to further refine decision-making capabilities.

1.2 Document conventions

- EDA (Exploratory Data Analysis) – The process of summarizing and visualizing data to extract meaningful patterns and insights.
- Web Scraping – The technique used to extract real estate data from online sources for analysis.
- Feature Engineering – The process of transforming raw data into meaningful features to improve machine learning model performance.
- Optuna – A hyperparameter optimization framework used for Bayesian Optimization in this project.
- Data Visualization – Graphical representations of real estate trends using libraries like Matplotlib, Seaborn, and Plotly.
- Regression Models – Machine learning algorithms used to predict property prices based on various features.

- Streamlit – A Python framework used to build an interactive web-based dashboard for showcasing real estate insights.

1.3 Scope

Real Estate Analytics is designed to be a lightweight yet powerful real estate data analysis tool that helps users extract, analyze, and visualize property trends in an efficient and structured manner. Unlike traditional real estate platforms that focus purely on listing properties, this project aims to bridge the gap between raw data and actionable insights by leveraging machine learning and data visualization to predict property prices and identify investment opportunities.

A key feature of this project is price prediction, where machine learning models analyze historical data to estimate property values based on factors like location, amenities, and market trends.

Additionally, the project provides three types of recommendations to assist users in making informed decisions:

1. Price-Based Recommendation – Suggests properties that offer the best value based on budget and market trends.
2. Facility-Based Recommendation – Filters properties based on user preferences such as schools, hospitals, and public transport accessibility.
3. Location-Based Recommendation – Identifies the best areas for investment based on demand, infrastructure, and future growth potential.

With these features, the project serves as a data-driven decision-making tool for buyers, investors, and real estate professionals. Future enhancements may include real-time data updates, AI-powered recommendations, and integration with rental market trends to further refine decision-making capabilities.

1.4 References

- GitHub Repository: [GitHub Repo]
- 99acres Website: Used as a primary source for real estate data collection through web scraping (99acres.com)
- Optuna Documentation: Reference for Bayesian Optimization in hyperparameter tuning (Optuna Docs)
- Matplotlib & Seaborn Documentation: Used for data visualization and graphical analysis (Matplotlib Docs, Seaborn Docs)
- Scikit-Learn Documentation: Reference for machine learning model implementation (Scikit-Learn Docs)
- Pandas & NumPy Documentation: Used for data preprocessing and analysis (Pandas Docs, NumPy Docs)

1.5 Overview

This document outlines the functional and non-functional requirements of the Real Estate Analytics project, providing a structured breakdown of the system. It serves as a guide for developers, data analysts, and stakeholders involved in the project.

The next sections will cover:

- Overall Description – Provides details on the product's purpose, user characteristics, assumptions, and constraints.
- Specific Requirements – Defines the system's functional, performance, and external interface requirements.
- Design Constraints – Describes any limitations related to the system's architecture, technology stack, or regulatory requirements.
- Appendices and References – Includes additional supporting materials and external references used for development.

2. Overall Description

1.6 Product Perspective

Real Estate Analytics is a data-driven web application developed using Python-based data science and machine learning frameworks. It serves as a standalone analytical tool, enabling users to extract, analyze, and visualize real estate trends efficiently. The project integrates web scraping, machine learning, and data visualization to provide insights into property prices, demand patterns, and investment opportunities.

Key Integrations:

- Frontend: Built with Streamlit, offering an interactive and user-friendly dashboard for real estate insights.
- Backend: Developed using Flask, handling data processing, ML model execution, and API requests.
- Database: Uses CSV or structured datasets obtained through web scraping for data storage and analysis.
- Data Processing: Leverages Pandas and NumPy for data cleaning, transformation, and feature engineering.
- Machine Learning: Implements regression models for price prediction and recommendation algorithms for location and facility-based suggestions.
- Visualization: Uses Matplotlib, Seaborn, and Plotly to generate interactive charts and real estate insights.
- Optimization: Incorporates Optuna for Bayesian Optimization to fine-tune ML model.

1.7 Product functions

Real Estate Analytics provides a range of functionalities to assist users in analyzing real estate data and making informed decisions:

- Real Estate Price Prediction: Uses machine learning to estimate property prices based on factors such as location, built-up area, and amenities.
- Interactive Data Visualization: Offers charts, graphs, and maps to explore real estate trends, price distributions, and sector-wise comparisons.
- Apartment Recommendation System: Provides recommendations based on price similarity, facilities, and location proximity, helping users find the best-suited properties.
- Sector-Wise Market Insights: Analyzes price trends over time, identifying high-demand areas and potential investment hotspots.
- User-Friendly Web Application: Built with Streamlit, allowing users to interact with data through an intuitive and responsive interface.
- Customizable Search & Filters: Users can refine results based on property type, budget, and preferred localities for a more personalized experience.

1.8 User classes and characteristics

Real Estate Analytics is designed to cater to multiple user groups:

- Homebuyers & Investors: Individuals looking to purchase property and seeking data-driven insights on pricing and market trends.
- Real Estate Agents & Consultants: Professionals needing a visual and analytical tool to assist clients in property selection and valuation.
- Data Analysts & Researchers: Users interested in exploring real estate data patterns, price fluctuations, and investment opportunities.
- Students & Enthusiasts: Individuals learning data science and real estate analytics, requiring a practical tool to analyze property trends.

User Characteristics

The Real Estate Analytics platform is designed for a diverse range of users with different levels of expertise in real estate and data analytics.

1. Technical Proficiency:

- Beginners: Homebuyers or investors with little to no experience in data analysis, relying on visual insights and recommendations.

- Intermediate Users: Real estate professionals or researchers with a basic understanding of data-driven decision-making.
- Advanced Users: Data analysts or ML practitioners who can interpret predictive models and optimize investment strategies.

2. Market Knowledge:
   - Homebuyers: Individuals looking to purchase a property and seeking price insights and locality comparisons.
   - Investors: People analyzing market trends for long-term property investments.
   - Real Estate Agents: Professionals using data-backed insights to assist clients in buying and selling properties.

3. User Expectations:
   - Accurate Price Prediction: Users expect reliable price estimates based on property attributes.
   - Easy Data Exploration: A user-friendly dashboard with visualizations for market trends.
   - Smart Recommendations: AI-powered property suggestions based on price, location, and facilities.

1.9 Operating environment

Real Estate Analytics is designed to operate in the following environments:

- **Client-Side:**
  - Accessible via modern web browsers such as Chrome, Firefox, and Edge.
  - Requires an internet connection to access real estate insights, predictions, and recommendations.
  - Optimized for both desktop and mobile viewing using Streamlit's responsive interface.
- **Server-Side:**
  - Python-based backend powered by Flask for handling requests and executing ML models.
  - Uses Pandas and NumPy for data processing and feature engineering.
  - Pre-trained ML models stored using Pickle for efficient price prediction.
  - CSV-based or database-driven data storage, depending on real estate dataset availability.
  - This ensures smooth performance and real-time analytical insights for users.

1.10 Design and implementation constraints

Several constraints influence the design and implementation of the Real Estate Analytics platform:

- Technology Stack: The project is built using Python, Flask, and Streamlit, limiting the choice of frameworks and requiring compatibility with machine learning libraries like Scikit-Learn and Optuna.
- Performance Requirements: The system must process large real estate datasets efficiently while ensuring real-time price prediction and visualization without significant delays.
- Data Availability: The accuracy of price predictions and recommendations depends on the quality and quantity of real estate data collected via web scraping or external sources.
- Security Considerations: Protection of user input data and secure handling of machine learning models to prevent unauthorized access or manipulation.
- Scalability: The platform should be capable of handling increasing data volumes and multiple concurrent users without compromising performance.

These constraints guide the system's architecture, ensuring efficiency, accuracy, and reliability.

1.11 Assumption and dependencies

The development and operation of Real Estate Analytics are based on the following assumptions and dependencies:

- User Access: Assumes users have access to modern web browsers and a stable internet connection to interact with the platform.
- Data Availability: The system relies on regular updates of real estate datasets through web scraping or external sources to maintain accurate predictions and recommendations.
- Third-Party Libraries: Depends on the continued support and functionality of Python libraries such as Pandas, NumPy, Scikit-Learn, Plotly, and Optuna for data analysis, visualization, and machine learning.
- Backend Services: Relies on the performance of Flask as the backend framework, along with Streamlit for UI rendering and Pickle for model storage and retrieval.
- Computational Resources: Assumes the hardware running the application can handle data processing, ML model inference, and real-time visualization without significant delays.
- These assumptions and dependencies are critical for ensuring the smooth operation of the system.

2. Specific requirements

2.1 Functional requirements

2.1.1 Price Prediction System

- The system shall allow users to input property details such as location, built-up area, number of rooms, and amenities.
- A machine learning model shall process the inputs and provide an estimated property

price range.

- The system shall ensure accurate predictions by utilizing historical data and feature engineering techniques.
- Users shall receive a detailed breakdown of how various factors influence the predicted price**.**

2.1.2 Data Visualization & Insights

- The system shall provide interactive charts, graphs, and maps to display real estate market trends.
- Users shall be able to explore price distributions, sector-wise price variations, and time-based trends.
- The platform shall offer a 3D visualization for analyzing price vs. built-up area vs. bedrooms.
- Users shall be able to filter data based on property type, locality, and price range.

2.1.3 Apartment Recommendation System

The system shall recommend apartments based on three similarity measures:

- Price Similarity – Suggests properties with a similar price range.
- Facility Similarity – Finds properties with matching amenities.
- Location Similarity – Recommends apartments near the selected location.

1. Users shall be able to adjust similarity weights to customize recommendations.
2. The system shall display recommendations with property details and external links for further exploration.

2.1.4 Database Management

- The system shall store real estate listings, user queries, and ML model outputs in a structured format.
- The system shall support efficient queries for fast retrieval and updates of property data.
- Data shall be regularly updated to ensure predictions and recommendations remain accurate.

2.1.5 Performance and Scalability

- The system shall be optimized for fast ML model inference and real-time data visualization.
- The application shall function smoothly across different devices and screen sizes without significant performance issues.
- The system shall be scalable to handle an increasing number of property listings, queries, and users.

2.2 External Interface Requirements

2.2.1 User Interfaces

- The system shall provide a web-based graphical user interface (GUI) for user interaction.
- The interface shall include interactive dashboards, charts, and maps for real estate analytics.
- Users shall be able to input property details, view price predictions, and explore recommendations through an intuitive UI.

2.2.2 Hardware Interfaces

- The system shall be compatible with standard input devices such as a mouse, keyboard, and touchscreens.
- The application shall run on any device with a web browser, including desktops, laptops, tablets, and mobile phones.
- The system shall support high-resolution display scaling for better visualization of property insights.

2.2.3 Software Interfaces

- The system shall use CSV files or a database to store real estate listings, user inputs, and ML model outputs.
- The backend shall be developed using Flask, handling API requests and ML model execution.
- The frontend shall be built using Streamlit, providing an interactive and dynamic user experience.
- The system shall integrate with third-party data sources or web scraping techniques to ensure up-to-date real estate information.

2.2.4 Communication Interfaces

- The system shall communicate over HTTPS to ensure secure data transmission between the user and the server.
- API endpoints shall follow RESTful principles for seamless data exchange between the frontend, backend, and machine learning models.
- The system shall support real-time data updates when integrating web scraping or external real estate sources.

2.3 Performance requirements

- The system shall provide real-time analytics and visualizations with minimal processing lag.
- The response time for price predictions and recommendations shall not exceed two seconds under normal operating conditions.
- The application shall efficiently handle large datasets and multiple concurrent users

without significant performance degradation.

- Machine learning models shall be optimized for fast inference, ensuring smooth user interaction.

2.4 Design constraints

- The system shall adhere to responsive design principles, ensuring usability across desktops, tablets, and mobile devices.
- The database or dataset structure shall be optimized for scalability and efficient querying, allowing for quick retrieval of property insights.

2.5 Software system attributes

2.5.1 Reliability and Availability

- The system shall be available 99% of the time, except during maintenance or updates.
- The dataset and model files shall be regularly backed up to prevent data loss.
- The price prediction and recommendation systems shall have error-handling mechanisms to ensure smooth functionality.

2.5.2 Security

- User interactions and inputs shall be secured to prevent data leaks or manipulation.
- API endpoints shall be protected against unauthorized access using authentication and access control measures.
- The system shall follow best practices to prevent injection attacks, data breaches, or tampering with predictions and recommendations.

2.5.3 Maintainability and Extensibility

- The system architecture shall be modular, allowing for future enhancements, such as AI-powered insights or additional filtering options.
- The codebase shall be well-documented, aiding future development, debugging, and collaboration.
- The application shall allow for seamless integration of new datasets or additional machine learning models as needed.

2.5.4 Portability

- The application shall be compatible with all major web browsers, including Chrome, Firefox, Edge, and Safari.
- The system shall be deployable on cloud platforms to ensure scalability and availability for a growing number of users.

3. Appendices

3.1 Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) provides a graphical representation of how data moves within the Real Estate Analytics system. It illustrates the interactions between users, databases, and system processes.

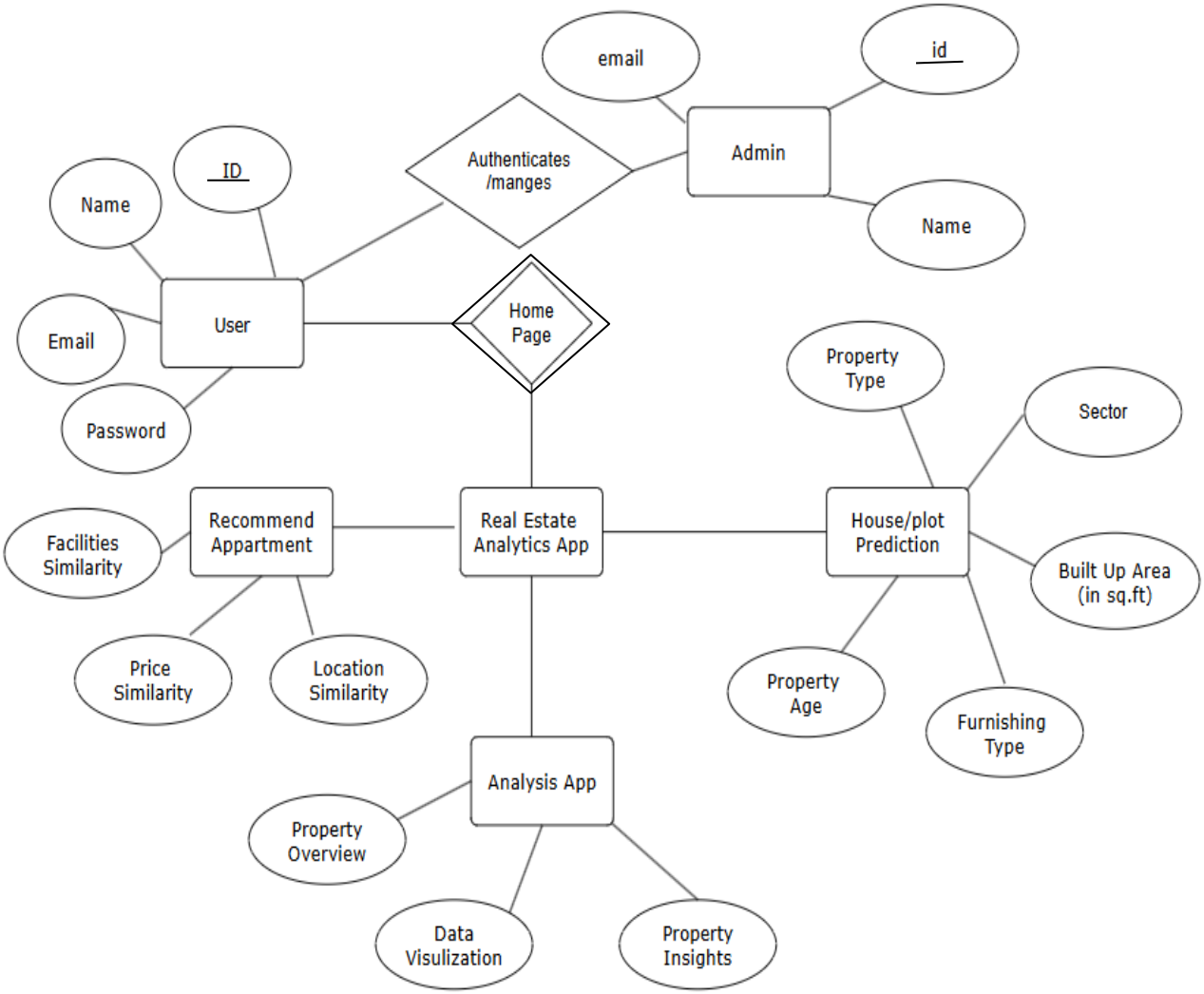
- **Level 0 (Context Diagram):** Represents the entire system as a single process interacting with users and external components such as data sources and ML model**s**.
- **Level 1 DFD:** Breaks down the system into multiple processes, including:

  1. **User Input Processing** – Accepting property details for prediction.
  2. **Machine Learning Model Execution** – Generating price predictions based on user inputs.
  3. **Data Visualization Module** – Creating charts, graphs, and real estate insights.
  4. **Recommendation Engine** – Suggesting apartments based on price, facilities, and location similarity.
  5. **Database Management** – Handling storage and retrieval of datasets.

3.2 Glossary

- **Price Prediction Model**: A machine learning model that estimates the value of a property based on various features.
- **Sector:** A geographical division used for categorizing properties in the real estate dataset.
- **Feature Engineering:** The process of transforming raw data into meaningful inputs for machine learning models.
- **Recommendation System:** An AI-based mechanism that suggests similar properties based on predefined similarity metrics.
- **Bayesian Optimization:** A hyperparameter tuning technique used to improve model accuracy.
- **Visualization Dashboard:** A web-based interface displaying charts, maps, and analytics for real estate trends.

Section 3: Entity-relationship Diagram



1. User
   - Attributes:
     - ID
     - Name
     - Email
     - Password
   - Actions/Relations:
     - Users can log in using credentials.
     - Can access home page features like property recommendations and analysis tools.
     - Input preferences are used to generate similar apartment suggestions.

2. Admin
   - Attributes:
     - id
     - Name
     - Email

- Actions/Relations:
    - Admins manage and authenticate users.
    - Responsible for image verification and backend operations.

3. Real Estate Analytics App
    - Central platform that connects modules for:
        - Recommendation
        - Prediction
        - Property Analysis

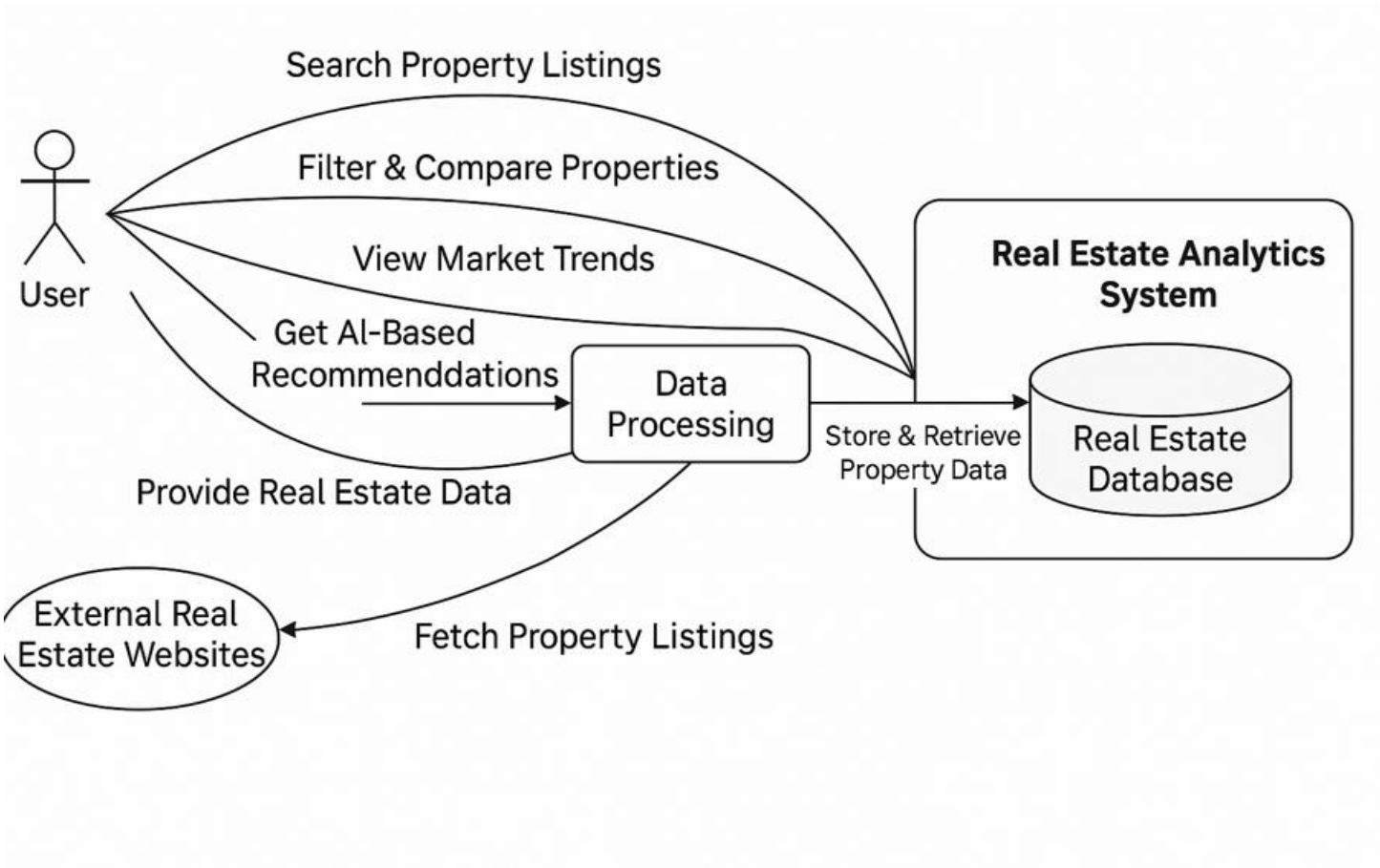
4. Recommend Apartment
    - Attributes/Factors:
        - Facilities Similarity
        - Price Similarity
        - Location Similarity
    - Actions/Relations:
        - Provides users with apartment suggestions based on similarity algorithms.
        - Integrated with the main app via user preferences.

5. House/Plot Prediction
    - Attributes:
        - Property Type
        - Sector
        - Built Up Area (in sq.ft)
        - Property Age
        - Furnishing Type
    - Actions/Relations:
        - Predicts house or plot prices using machine learning models.
        - Takes structured input for accurate price prediction.

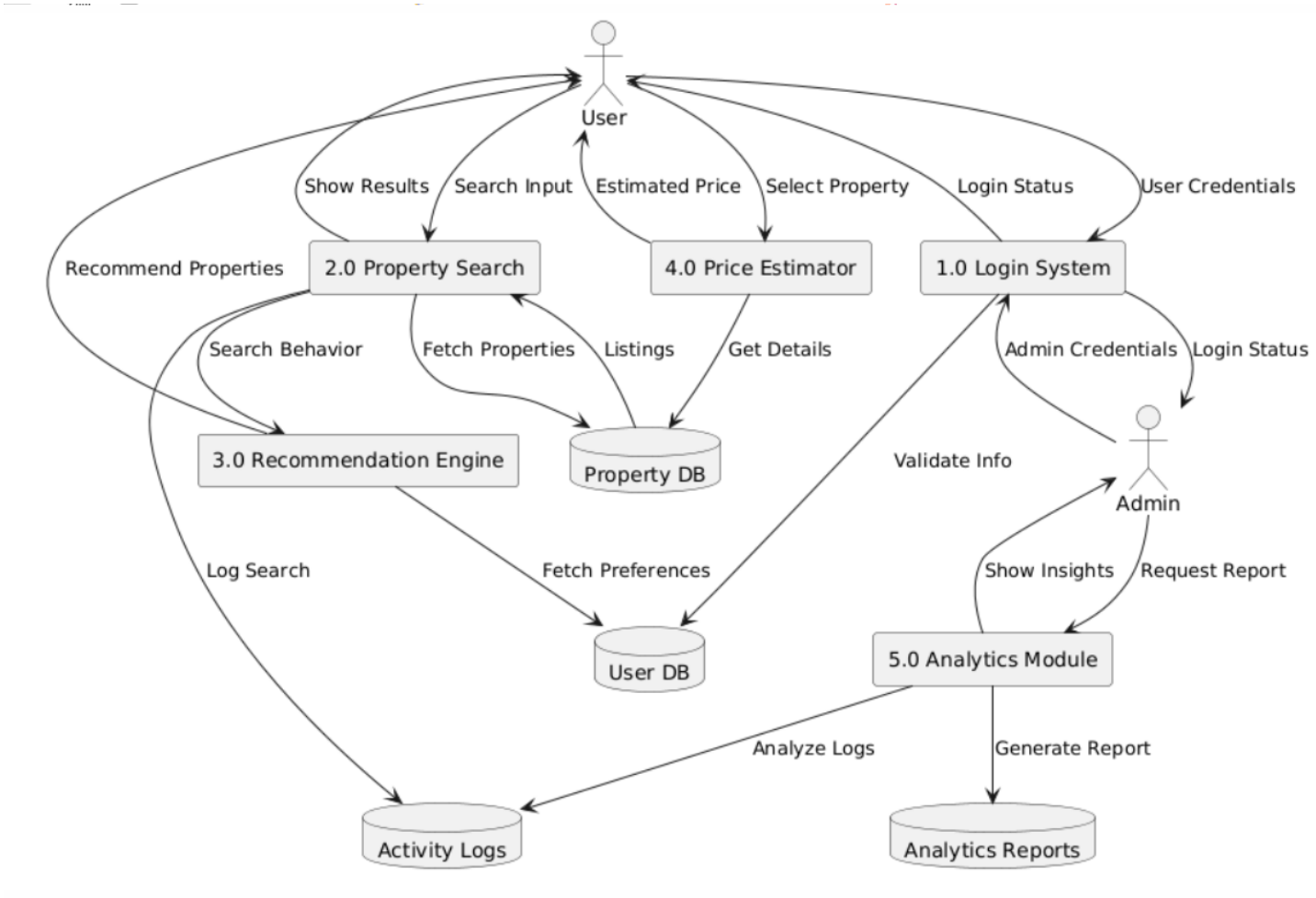
6. Analysis App
    - Attributes/Sections:
        - Property Overview
        - Data Visualization
        - Property Insights
    - Actions/Relations:
        - Presents EDA dashboards and market insights.
        - Helps users understand trends and property features.

7. Relationships and Interactions
    - User ↔ Home Page
        - After login, users are directed to the Home Page to access app functionalities.
    - Home Page ↔ Recommendation & Prediction Modules
        - Users can explore recommended apartments or predict house/plot prices.
    - Analytics App ↔ Main System
        - Offers property data analysis, linked to the prediction engine and stored datasets.
    - Admin ↔ Users
        - Admins authenticate and manage user accounts securely.

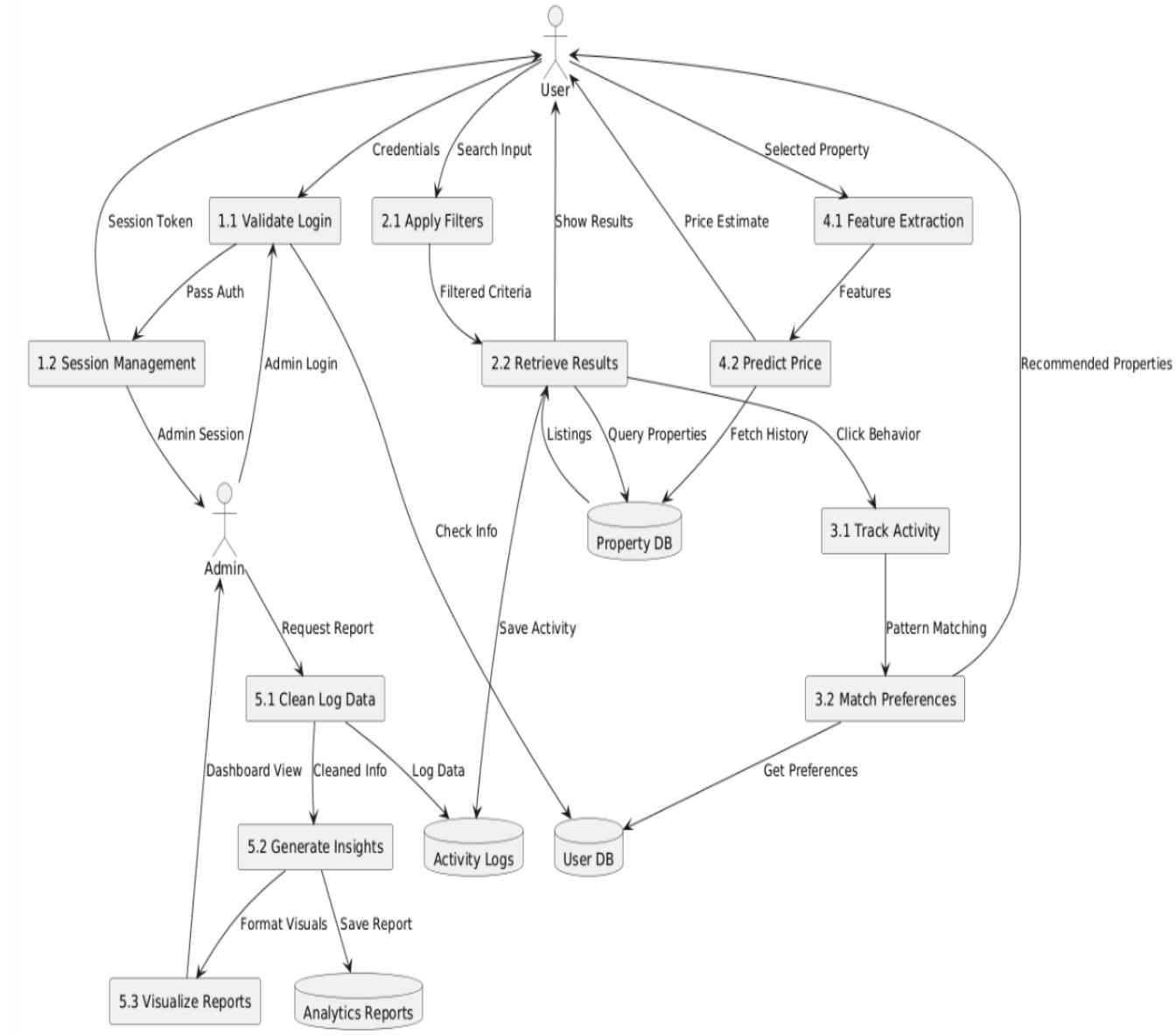
Section 4: Data Flow Diagram (DFD)



Level 0 DFD

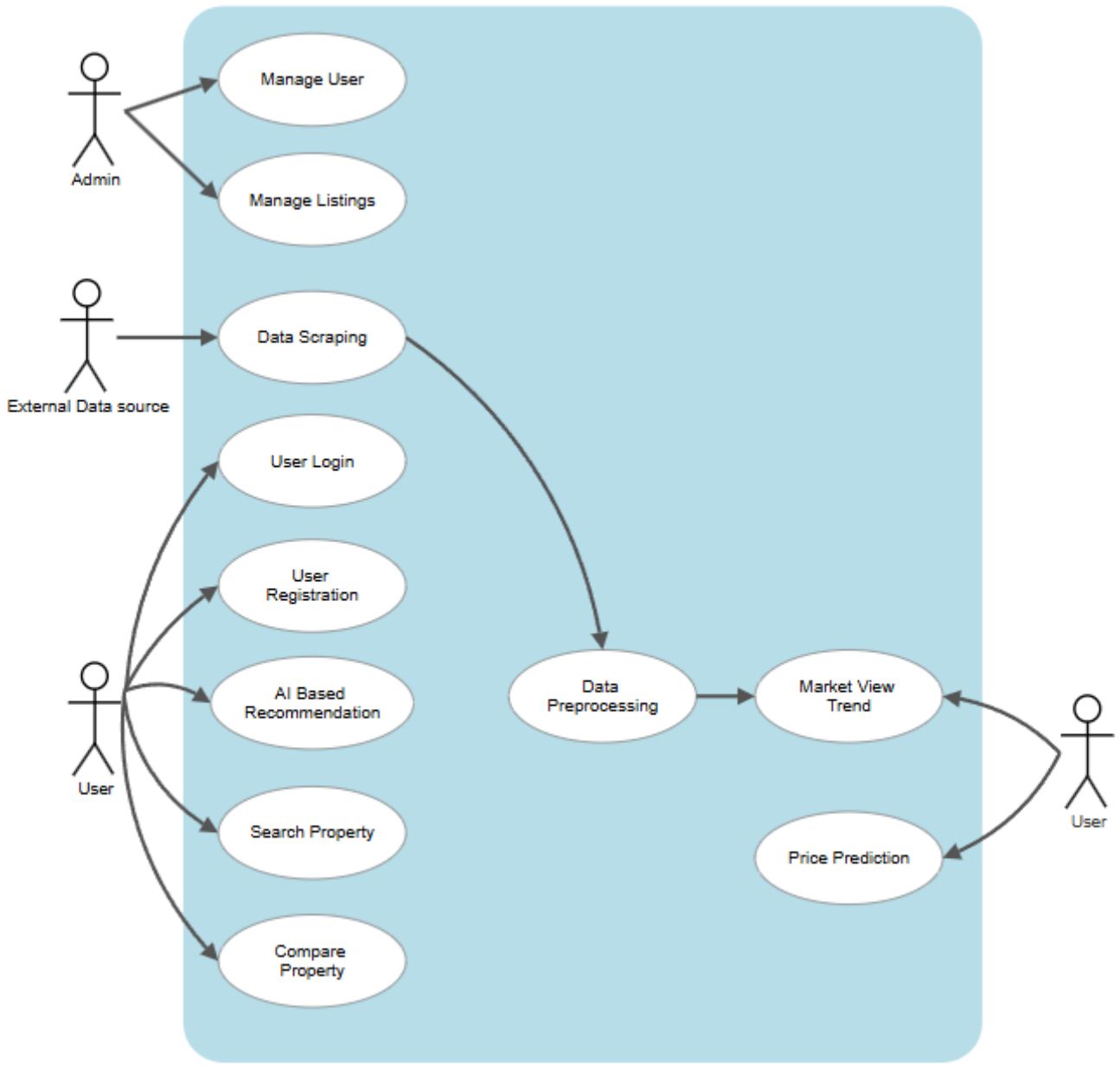

User
Show Results
Search Input
Estimated Price
Select Property
Login Status
User Credentials
Recommend Properties
2.0 Property Search
4.0 Price Estimator
1.0 Login System
Search Behavior
Fetch Properties
Listings
Get Details
Admin Credentials
Login Status
3.0 Recommendation Engine
Property DB
Validate Info
Admin
Log Search
Fetch Preferences
Show Insights
Request Report
User DB
5.0 Analytics Module
Analyze Logs
Generate Report
Activity Logs
Analytics Reports

Level 1 DFD


User
Credentials
Search Input
Selected Property
Session Token
1.1 Validate Login
2.1 Apply Filters
Show Results
Price Estimate
4.1 Feature Extraction
Pass Auth
Filtered Criteria
Features
1.2 Session Management
Admin Login
2.2 Retrieve Results
4.2 Predict Price
Recommended Properties
Admin Session
Listings
Query Properties
Fetch History
Click Behavior
Check Info
Property DB
3.1 Track Activity
Admin
Request Report
Save Activity
Pattern Matching
5.1 Clean Log Data
3.2 Match Preferences
Dashboard View
Cleaned Info
Log Data
Get Preferences
5.2 Generate Insights
Activity Logs
User DB
Format Visuals
Save Report
5.3 Visualize Reports
Analytics Reports

Level 2 DFD

Section 5: Use case diagram



1. Manage User

Brief Description: This use case describes how an Admin manages users within the system.
Actors:

- Admin

Basic Flow:

1. The admin navigates to the user management section.

2. The system displays a list of registered users.
3. The admin selects a user to update, deactivate, or delete.
4. The system processes the request and updates the database.
5. A confirmation message is displayed.

Alternative Flow:

- If an invalid action is selected, an error message is displayed.

Pre-Conditions:

- The admin must be logged in.

Post-Conditions:

- User data is updated or removed from the database.

2. Manage Listings

Brief Description: This use case describes how an Admin manages real estate property listings.

Actors:

- Admin

Basic Flow:

1. The admin accesses the property listings section.
2. The system displays all available listings.
3. The admin can add, update, or remove property details.
4. The system saves the changes and displays a confirmation message.

Alternative Flow:

- If the listing update fails, the system prompts an error message.

Pre-Conditions:

- The admin must be logged in.

Post-Conditions:

- Property listings are successfully modified in the system.

3. Data Scraping

Brief Description: This use case describes how External Data Sources provide data for property listings.

Actors:

- External Data Source

Basic Flow:

1. The system fetches real estate data from external sources.
2. The system processes and stores the data in a structured format.
3. The system makes the data available for further processing.

Alternative Flow:

- If the data retrieval fails, the system retries the operation.

Pre-Conditions:

- The system must have access to external real estate data sources.

Post-Conditions:

- New data is successfully stored in the database.

4. User Registration

Brief Description: This use case describes how a User registers on the platform.
Actors:

- User

Basic Flow:

1. The User navigates to the registration page.
2. The system prompts for name, email, and password.
3. The User enters the required details.
4. The system validates and creates an account.
5. A confirmation message is displayed.

Alternative Flow:

- If the entered data is invalid, an error message is shown.

Pre-Conditions:

- The User must provide a valid email.

Post-Conditions:

- The new account is created and stored in the database.

5. User Login

Brief Description: This use case describes how a User logs into the system.
Actors:

- User

Basic Flow:

1. The User enters their email and password on the login page.
2. The system verifies the credentials.
3. Upon successful verification, the user is redirected to the dashboard.

Alternative Flow:

- If authentication fails, an error message is displayed.

Pre-Conditions:

- The User must have an existing account.

Post-Conditions:

- The User is logged in successfully.

6. AI-Based Recommendation

Brief Description: This use case describes how the system recommends properties based on AI.
Actors:

- User

Basic Flow:

1. The User searches for properties.
2. The system analyzes user preferences using AI.
3. The system suggests properties based on historical data and user interests.
4. The recommendations are displayed to the user.

Alternative Flow:

- If no relevant recommendations are found, the system displays generic property listings.

Pre-Conditions:

- The system must have sufficient data to generate recommendations.

Post-Conditions:

- The User receives personalized property suggestions.

7. Search Property

Brief Description: This use case describes how users search for properties.

Actors:

- User

Basic Flow:

1. The User enters search criteria (location, price, type, etc.).
2. The system retrieves and displays matching properties.

Alternative Flow:

- If no properties match, a “No Results Found” message is displayed.

Pre-Conditions:

- The database must have property listings.

Post-Conditions:

- Search results are displayed to the User.

8. Market View Trend Analysis

Brief Description: This use case describes how the system analyzes real estate market trends.

Actors:

- System

Basic Flow:

1. The system extracts property price data over time.
2. The system generates trend graphs and insights.
3. The User views market trends on the dashboard.

Alternative Flow:

- If no sufficient data is available, a message is displayed.

Pre-Conditions:

- The system must have historical property data.

Post-Conditions:

- Market trends are successfully visualized.

9. Price Prediction

Brief Description:This use case describes how the system predicts future property prices.
Actors:

- System
- User

Basic Flow:

1. The system analyzes historical pricing data.
2. The system applies machine learning algorithms.
3. The predicted prices are displayed to the User.

Alternative Flow:

- If prediction confidence is low, a warning message is displayed.

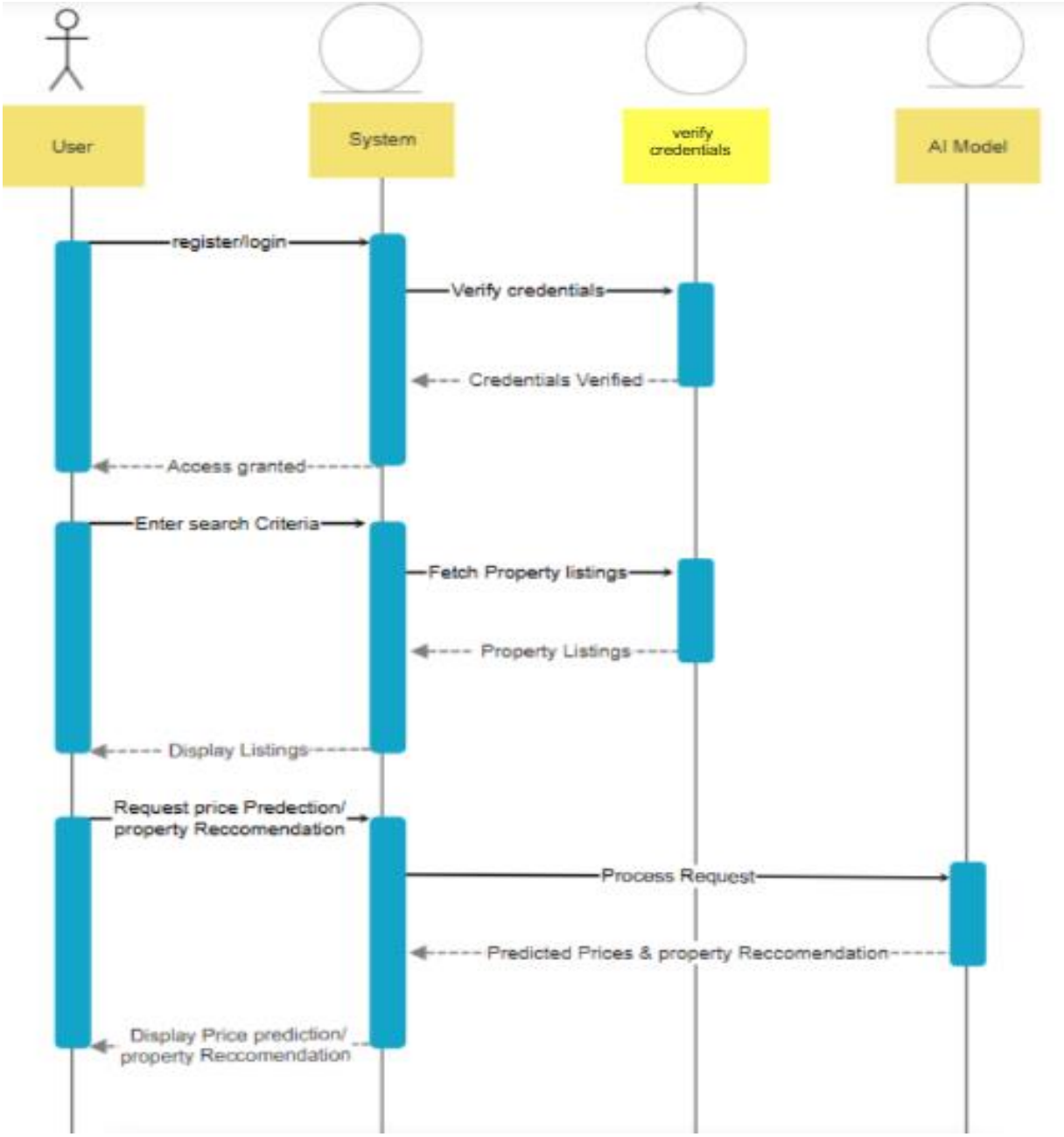
Pre-Conditions:

- The system must have a trained AI model.

Post-Conditions:

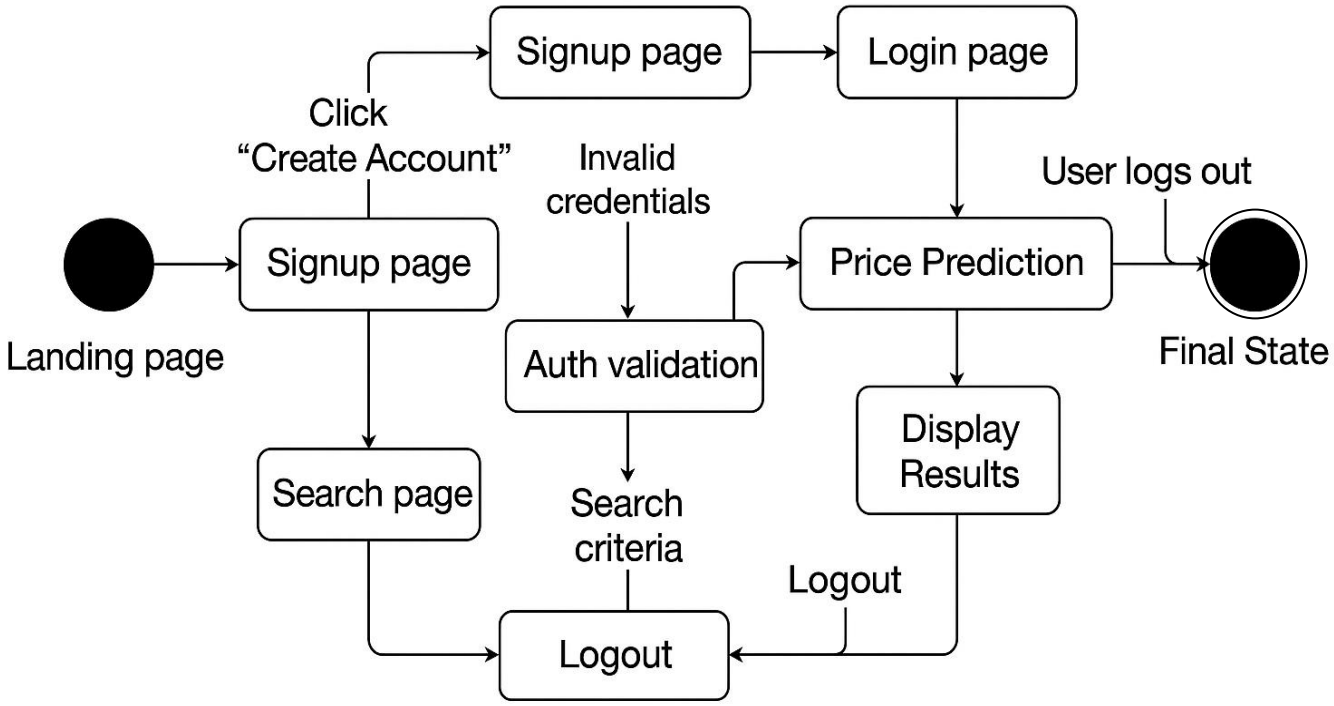
- Future price trends are shown to the User.

Section 6: Sequence Diagram



1. User Login/Register Initiation
    - Action:
        - The user starts by selecting register/login via the system interface.
2. Credential Verification
    - Action:
        - The system sends the credentials to the verification module.
        - Credentials are checked.
    - Outcome:
        - If correct → "Credentials Verified"
        - Access is granted to the user.
3. Property Search
    - Action:
        - User enters search criteria (e.g., location, price range).
        - System requests property listings from the backend.
    - Outcome:
        - Listings are returned to the system and shown to the user.
4. Request for Prediction/Recommendation
    - Action:
        - User requests price prediction or property recommendation.
        - The system forwards the request to the AI model.
5. AI Model Processing
    - Action:
        - AI model processes the input.
        - Returns predicted prices and/or recommended properties to the system.
6. Final Display
    - Action:
        - System displays the prediction and recommendation results to the user.

Section 7 : State-chart diagram



1. Initial State:
   - The system begins in the initial state, shown as a filled black circle.
   - This represents the starting point of the user's interaction with the Property Analytics system.

2. Landing Page State (Idle State):
   - From the initial state, the system transitions to the Landing Page.
   - This is the idle or waiting state where the user chooses to either create a new account or log in.
   - The user can:
     - Click "Create Account" to go to the Signup Page.
     - Click "Login" to go to the Login Page.

3. Signup Page State:

- If the user selects to create a new account, the system transitions to the Signup Page.
- After successful signup, the system moves to the Authentication Validation state to verify the credentials.

4. Authentication Validation State:

- The system checks the user's signup or login credentials.
- If the credentials are invalid, the system returns to the Signup or Login Page to retry.
- If the credentials are valid, the user is granted access and redirected to the Price Prediction page.

5. Login Page State:

- The user enters login credentials (email and password).
- The system transitions to the Authentication Validation state.
- Depending on the result:
    - Invalid credentials → Stay on Login Page to retry.
    - Valid credentials → Move to the Price Prediction page.

6. Price Prediction State (Core Functionality):

- Once logged in, users reach the Price Prediction interface.
- Here, users can:
    - Enter property search criteria.
    - Initiate price predictions using AI.
    - View results.
    - Logout.

7. Search Page State:

- From the Price Prediction page, users can enter or update search criteria.
- Once submitted, the system moves to Display Results.

8. Display Results State:

- The system processes the request using AI and displays the predicted pricing results.
- Users can:
    - Continue refining search,
    - Review results,
    - Or proceed to logout.

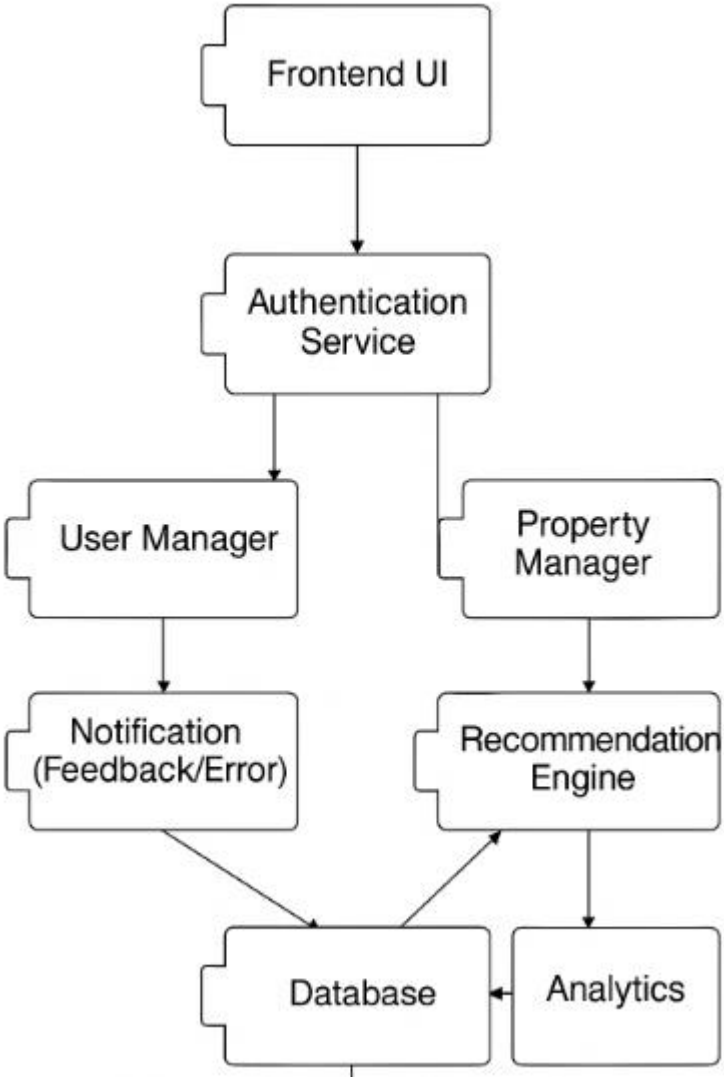
9. Logout State:

- At any point, users can choose to logout.
- Upon logout, the system transitions to the Final State.

10. Final State:

- This state is shown as a filled circle with a border.
- The system reaches this state when:
    - The user logs out,
    - Or after search and result viewing is complete,
    - Or when invalid login attempts end the session.

Section 8: component diagram



1. Frontend UI
   - This is the user-facing interface built using technologies like React.js or similar frameworks.
   - Users interact with this component for:
     - Logging in / signing up
     - Searching properties
     - Viewing price predictions
     - Receiving recommendations and feedback

2. Authentication Service

- Handles all user authentication and authorization logic.
- Validates credentials when a user tries to log in or sign up.
- Interfaces with the User Manager and Property Manager to grant access to user-specific or data-related functionalities.

3. User Manager

- Manages user-related operations such as:
  - Profile data
  - Preferences
  - Session data
- Passes necessary messages (success/failure) to the Notification component.
- Updates and retrieves data from the Database.

4. Property Manager

- Deals with the property listings, filtering, and search criteria input by users.
- Interacts with the Recommendation Engine to provide smarter, personalized property suggestions.
- Connects with the Database for reading property data.

5. Notification (Feedback/Error)

- Sends out feedback to the UI:
  - Success messages (e.g., “Search completed”)
  - Errors (e.g., “Invalid credentials”)
- Uses responses from the User Manager and system states to craft messages.
- Has access to Database to retrieve system status or logs if needed.

6. Recommendation Engine

- Uses search patterns, user behavior, and property features to recommend relevant listings.

- Interfaces with:
  - Property Manager for listing data
  - Analytics for behavior patterns

    - Database to fetch/store recommendation data

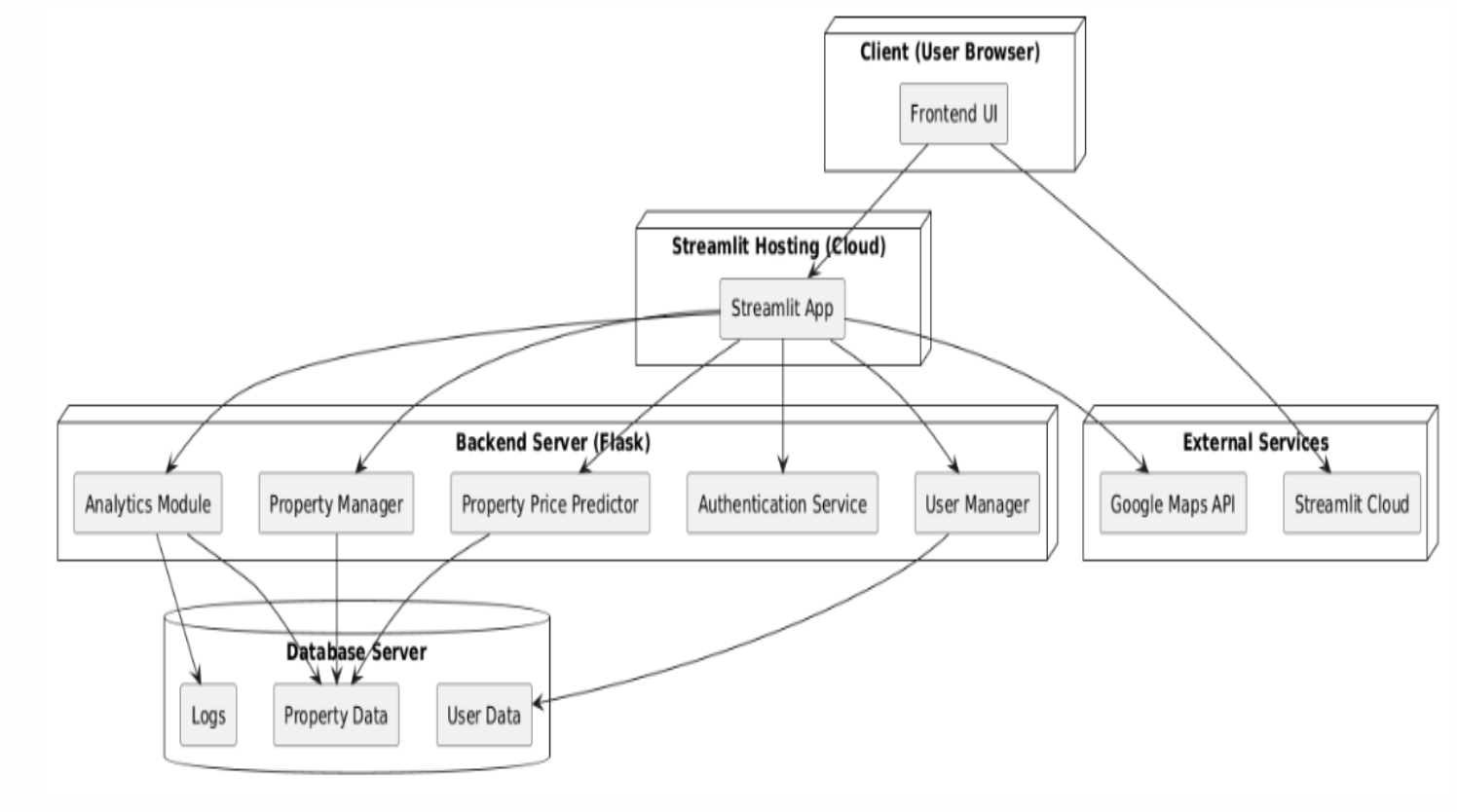
7. Analytics

- Analyzes:
    - User behavior (search history, clicks)
    - Property trends (most viewed, budget distribution)
- Feeds data into the Recommendation Engine to enhance personalization.
- Pulls raw data from the Database.

8. Database

- Centralized data repository for the entire system.
- Stores:
    - User credentials and profiles
    - Property listings
    - Search and recommendation history
    - Analytics data
- Accessed by almost all backend services to read/write data efficiently.

Section 9: Draw the Deployment diagram



This deployment diagram illustrates how different software modules of the Property Insight Platform are physically deployed across various infrastructure components including user devices, backend servers, and external services. It clearly outlines runtime environments, responsibilities, and communication paths between system components.

1. Client (User Browser)

• Component:

- Frontend UI: Built using Streamlit, hosted on the cloud for interactive user access.

• Responsibilities:

- Provides a clean and responsive interface for users to:
  - Search and filter properties
  - View analytics and predictions
  - Sign in / Register securely
- Sends HTTP requests to the backend for data and predictions.
- Displays dynamic insights returned from backend services.

2. Streamlit Hosting (Cloud)

- Component:
  - Streamlit App: Hosts the main frontend application and manages user interactions.
- Responsibilities:
  - Acts as a bridge between the user interface and backend APIs.
  - Sends user inputs to the Flask backend for processing and predictions.
  - Displays property analytics, prediction results, and user data.

3. Backend Server (Flask)

- Components:
  - Property Manager: Handles CRUD operations related to property listings.
  - Property Price Predictor: ML model serving module that predicts property prices based on input features.
  - Authentication Service: Manages login, signup, and session validation.
  - User Manager: Handles user profiles, preferences, and history.
  - Analytics Module: Performs data aggregation and visualization logic for trends and statistics.
- Responsibilities:
  - Processes all incoming API requests from the Streamlit UI.
  - Performs ML-based predictions and sends results to the frontend.
  - Communicates with the database for property, user, and analytics data.
  - Handles user authentication and session management.

4. Database Server (Cloud – MongoDB, PostgreSQL, or equivalent)

- Components:
  - Property Data: Stores listings, features, location, and pricing info.
  - User Data: Includes account information, saved properties, and interaction logs.
  - Logs: Tracks system events, errors, and predictions for auditing and monitoring.
- Responsibilities:
  - Acts as the central data store for the platform.
  - Provides scalable, persistent storage.
  - Serves as the backend for analytics and model training data.

5. External Services

- Components:
    - Google Maps API: Fetches location-based data and renders property locations.
    - Streamlit Cloud: Hosts the frontend UI securely and at scale.
- Responsibilities:
    - Enhances functionality via 3rd-party APIs (e.g., for maps and location data).
    - Ensures global accessibility of the UI without self-managed hosting.

6. Communication Flow

1. User opens the Streamlit web app in a browser and interacts with the UI.
2. User requests (e.g., price prediction, filter properties) are sent from Streamlit App to the Flask backend.
3. The Backend Server:
    - Validates users via the Authentication Service.
    - Routes requests to appropriate modules like:
        1. Property Price Predictor for ML results.
        2. Property Manager for listing operations.
        3. Analytics Module for data visualization.
4. Results are sent back up to the Streamlit App, which renders the output to the user in real time.

Section 10: Screenshots of the project

Module 1: Property Price Prediction

Description:

The Property Price Prediction module is designed to forecast the estimated market price of a property based on its key features such as location, area (sq. ft.), number of bedrooms, bathrooms, furnishing status, and amenities.

This module utilizes machine learning regression algorithms to analyze the relationship between various property attributes and their corresponding prices, providing accurate and data-driven predictions.

Objective:

To develop a predictive model capable of estimating real estate property prices with high accuracy using historical data scraped from 99acres.com.

Approach:

1. Data Collection & Preprocessing:
   - Web scraping was used to collect property listing data from 99acres.com.
   - Data cleaning involved handling missing values, removing duplicates, and encoding categorical variables.
   - Feature scaling and normalization were performed to enhance model performance.

2. Feature Engineering:
   - Extracted features such as price per square foot and location averages.
   - Performed correlation analysis to identify the most significant predictors of price.

3. Model Development:
   - Several regression models were trained and compared, including:
     - Linear Regression
     - Random Forest Regressor
     - XGBoost Regressor
   - The Random Forest model provided the best performance with optimal accuracy and low error.

4. Evaluation Metrics:
   - R^2 Score, Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE) were used for model evaluation.

   - Hyperparameter tuning was performed using Optuna to improve results.

5. Deployment:
   - The trained model was deployed on Streamlit, allowing users to input property details and instantly get price predictions.
   - The UI dynamically displays the predicted price based on user input fields.

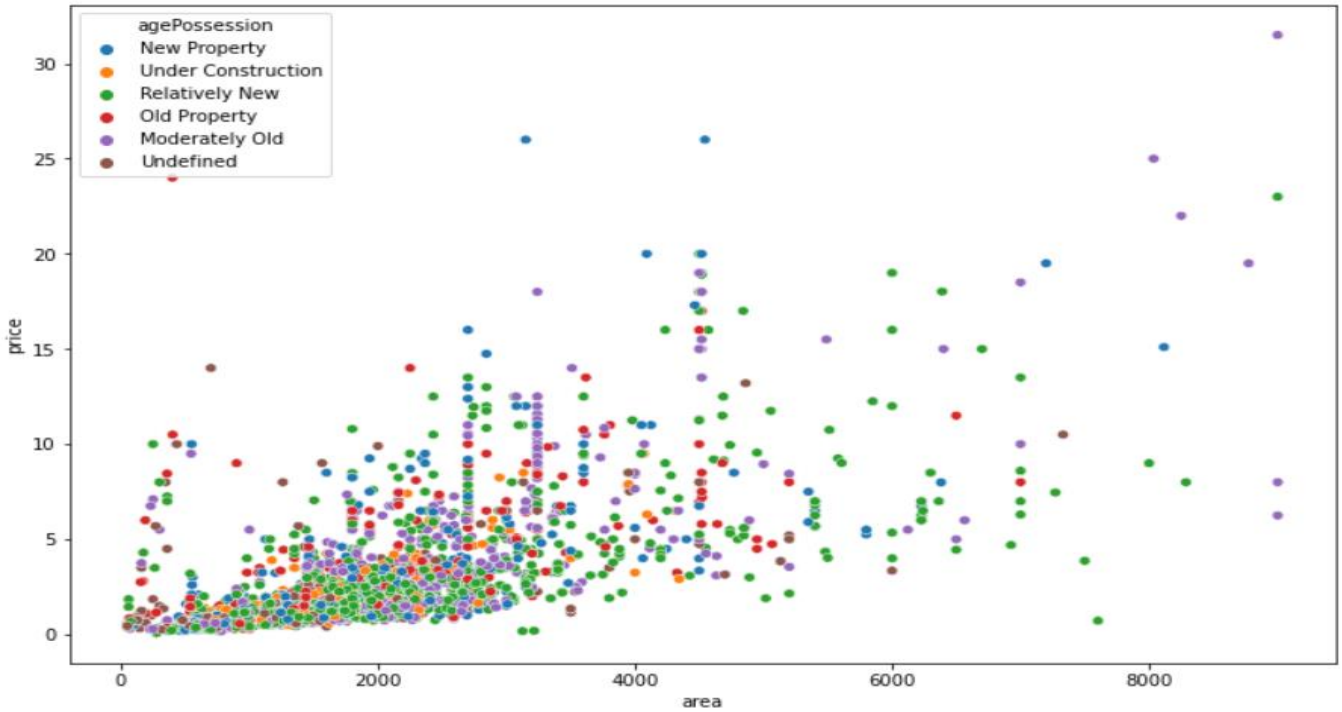


Figure 1 Data Preprocessing

	name	r2	mae
5	random forest	0.900529	0.453631
6	extra trees	0.902299	0.457174
10	xgboost	0.900643	0.483409
7	gradient boosting	0.889074	0.509390
4	decision tree	0.830704	0.543814
9	mlp	0.852892	0.629715
8	adaboost	0.820024	0.681451
0	linear_reg	0.829522	0.713011
2	ridge	0.829536	0.713523
1	svr	0.782917	0.818851
3	LASSO	0.059434	1.528906

Figure 2 Model Selection

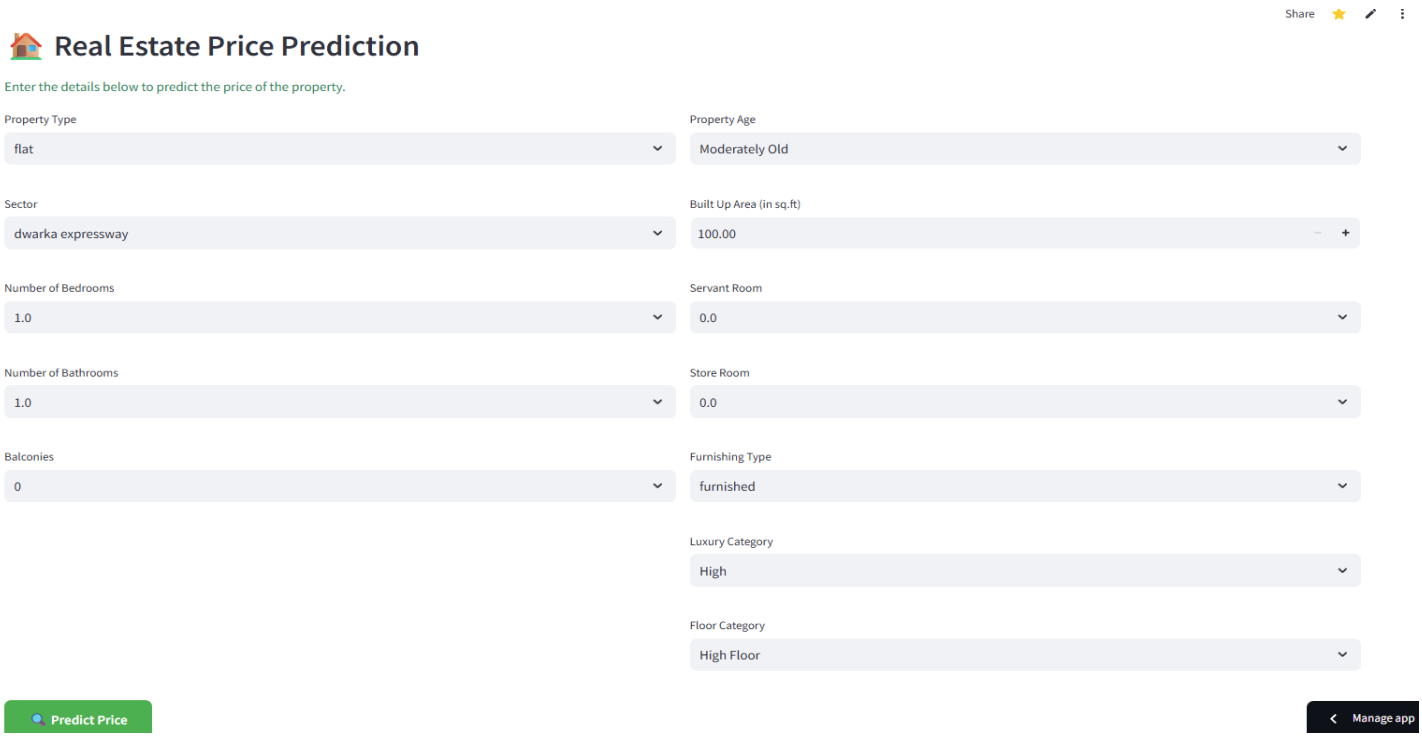


Figure 3 Streamlit User Interface for Price Prediction

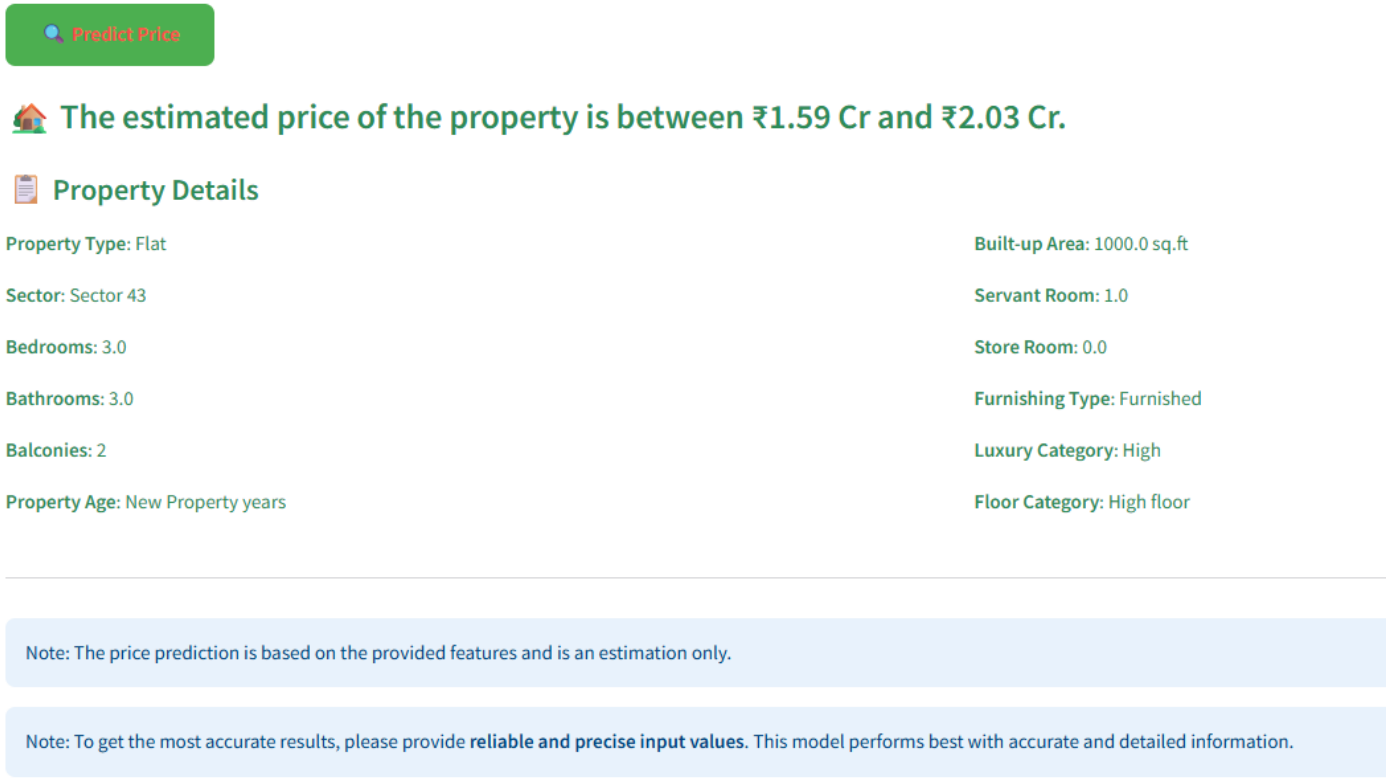


Figure 4 Predicted Property Price Output

Module 2: Analytics & Visualization

Description:
The Analytics & Visualization module provides interactive insights and visual dashboards to help users better understand the real estate market.
It analyzes large volumes of property data to uncover trends, patterns, and price distributions across various locations, furnishing types, and property sizes.
This module enables users to make informed decisions by visualizing data through charts, graphs, and summary dashboards.

Objective:
To create a data-driven analytical interface that allows users to explore property trends and market insights interactively using visualizations.

Approach:

1. Exploratory Data Analysis (EDA):
    - Conducted an in-depth analysis of property prices, locations, and configurations.
    - Detected outliers, missing data, and inconsistencies to improve dataset quality.
    - Used statistical summaries and correlation matrices to identify key influencing features.

2. Visualization Development:
    - Implemented visualizations using Matplotlib, Seaborn, and Plotly for interactive insights.
    - Created multiple graphs and dashboards to highlight patterns such as:
        - Average property price by location
        - Furnishing type vs. price relationship
        - Bedroom count vs. price correlation
        - Price distribution and area trends

3. Interactive Dashboard (Streamlit Integration):
    - Integrated visualizations within the Streamlit web app to provide a user-friendly and dynamic interface.
    - Added interactive filters allowing users to view data by city, property type, or budget range.
    - The dashboard automatically updates charts based on user selection.

4. Insights Derived:
    - Properties in premium areas showed higher average prices per square foot.
    - Furnished and semi-furnished properties tended to have better resale value.
    - Larger properties didn’t always imply higher price per sq. ft., showing localized market variations.

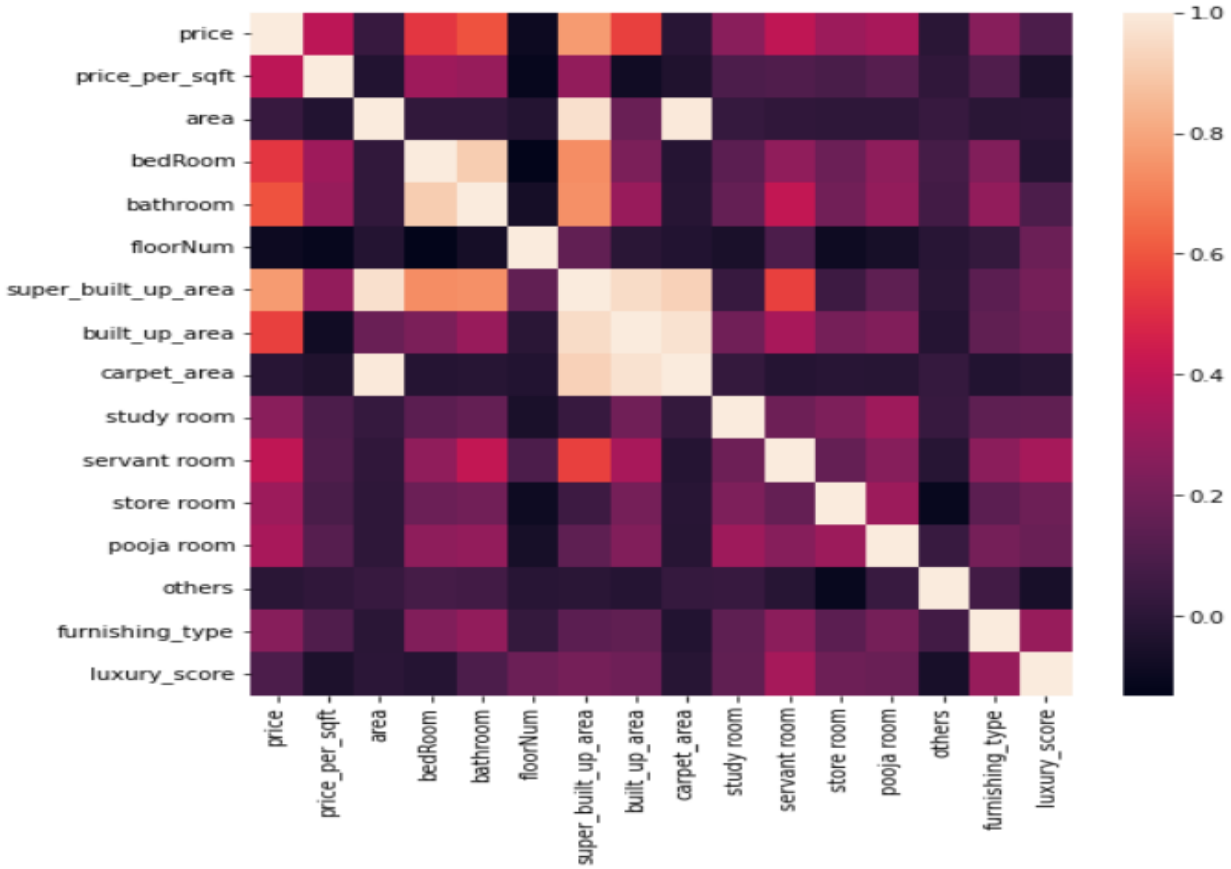


Figure 5 Correlation Heatmap of Property Features

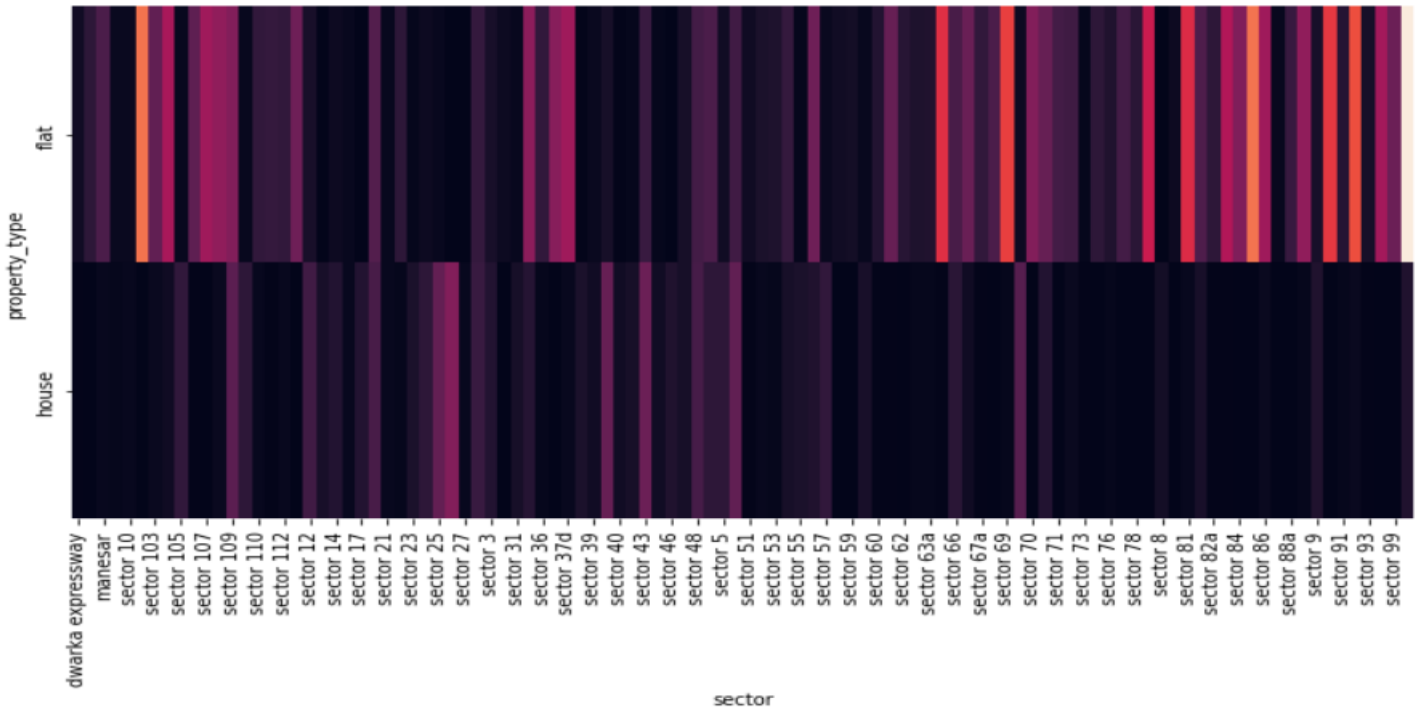


Figure 6 Heatmap average prices vary across major localities

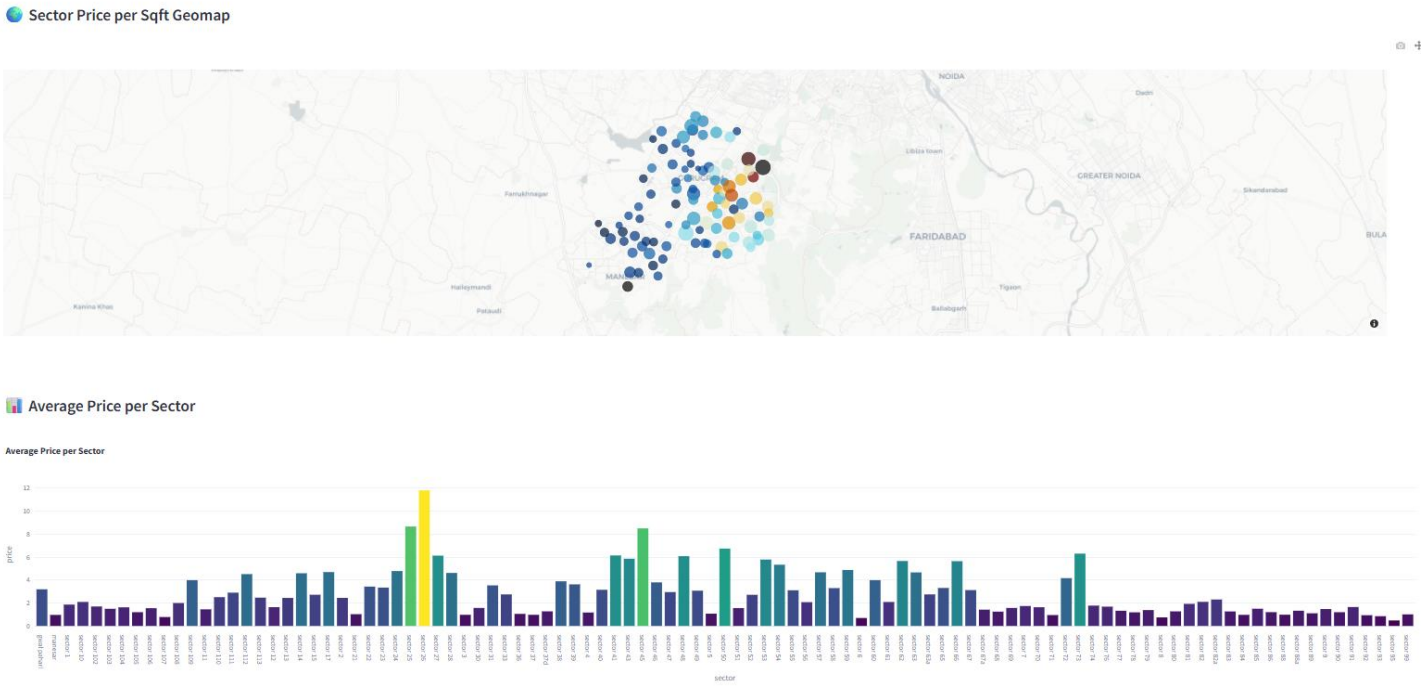


Figure 7 Depicts user interface with dynamic filters

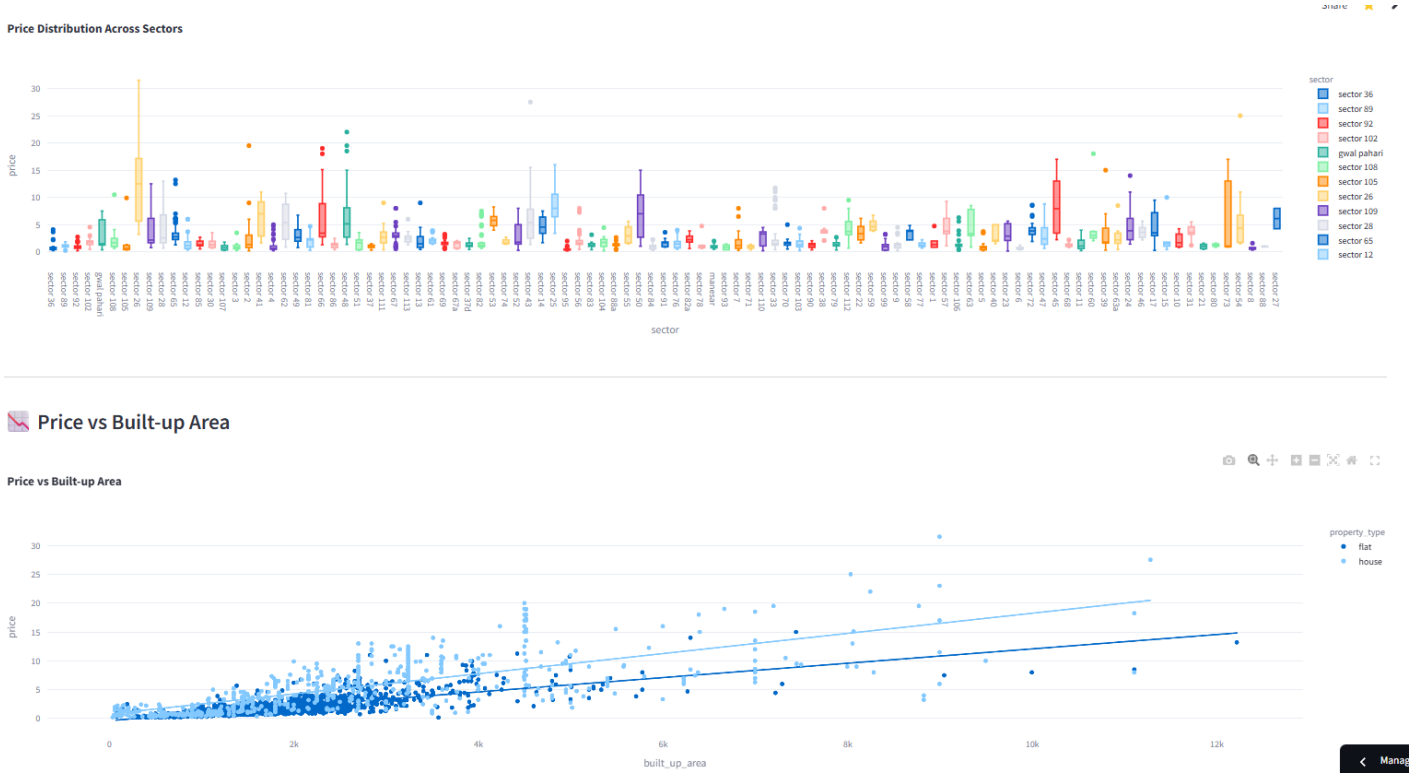


Figure 8 Visual representation of price range

Module 3: Property Recommendation System

Description:

The Property Recommendation System module is designed to provide users with personalized property suggestions based on their preferences and similarity measures.

It combines content-based filtering and spatial search techniques to recommend the most relevant properties.

The module includes two major components:

1. Adjustable Similarity Weights — allows users to tune the importance of different features like pricing, location, and facilities.
2. Find Nearby Apartments — enables users to discover apartments within a specified radius around a chosen location.

Objective:

To build an intelligent recommendation engine that helps users find the most suitable properties based on their desired features and proximity preferences.

Approach:

1. Adjustable Similarity Weights
   - Implemented a multi-factor similarity engine using cosine similarity to compute how closely each property matches the user's requirements.
   - Users can adjust similarity weights for:
     - Facilities Similarity (amenities, furnishing, etc.)
     - Pricing Similarity (budget alignment)
     - Location Similarity (geographical closeness)
   - Each slider ranges from 0.00 to 1.50, allowing flexible prioritization of specific factors.
   - The system recalculates similarity scores dynamically and displays the top recommended properties based on the chosen weight distribution.

2. Find Nearby Apartments
   - This feature helps users find apartments located near a specific area within a user-defined radius.
   - By selecting a location and specifying a radius (1–50 km), the system filters properties within that distance using geospatial coordinates.
   - This ensures users can discover nearby apartments that match their location preference and budget range.

3. Recommendation Logic
   - Property features (price, area, furnishing, amenities, location) are converted into vector form.
   - Cosine similarity is calculated between user input and each property vector.
   - Final recommendations are ranked based on the combined similarity score derived from the weighted features.
   - Results are displayed interactively in Streamlit with property details, estimated prices, and proximity indicators.

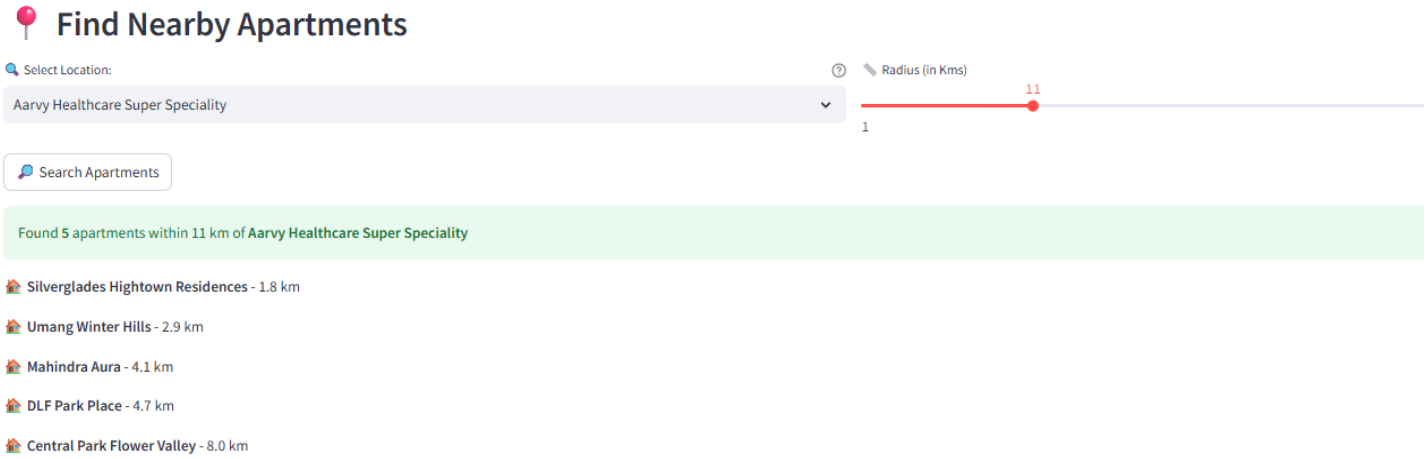


Figure 9 Find Nearby Apartments Interface

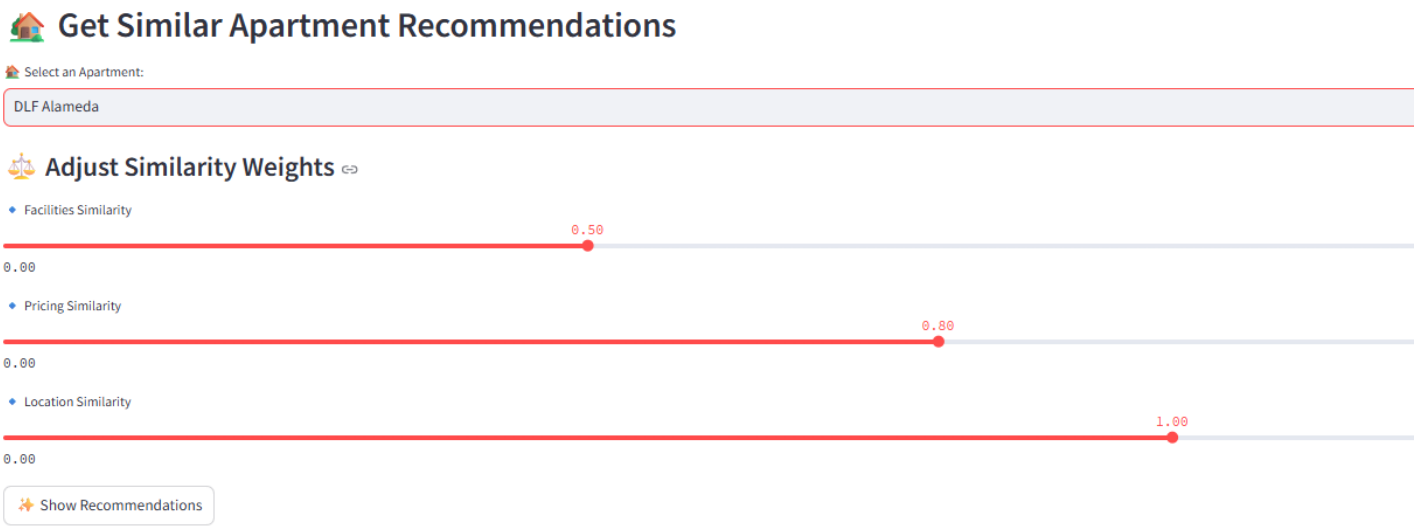


Figure 10 Adjustable Similarity Weights Interface

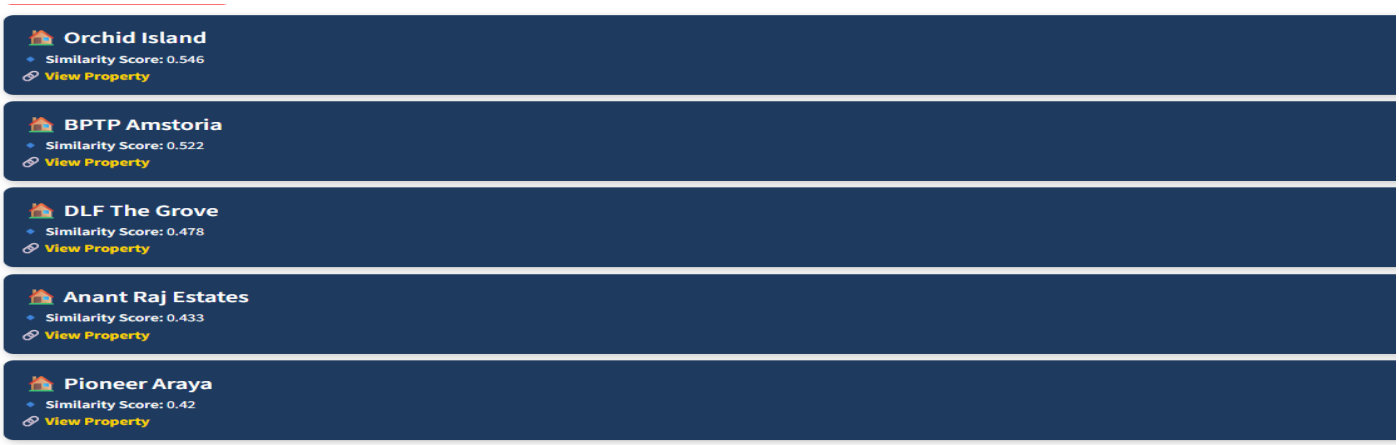


Figure 11 Recommended Apartments

Conclusion:

The Real Estate Analytics project successfully integrates data analysis, machine learning, and recommendation techniques to deliver a comprehensive solution for property insights and decision-making.

Through the three core modules — Price Prediction, Analytics & Visualization, and Property Recommendation — the system provides accurate price forecasts, market trend analysis, and personalized property suggestions.

The platform's interactive Streamlit interface enhances user experience by allowing real-time data exploration and dynamic visualization.

By leveraging advanced algorithms such as Random Forest Regression, XGBoost, and Cosine Similarity, the system demonstrates strong analytical accuracy and scalability.

Overall, this project bridges the gap between raw property data and actionable insights, empowering users, investors, and real estate professionals to make data-driven decisions with confidence.

Future Scope:

While the current system performs efficiently, there are several areas where it can be further enhanced to achieve greater accuracy and usability:

1. Integration of Real-Time Data:
   Incorporate APIs from real estate platforms to fetch live property listings and market updates for real-time analysis and prediction.
2. Advanced Deep Learning Models:
   Utilize deep learning models such as LSTM or Transformer-based architectures for improved price forecasting using temporal and textual data.
3. Geo-Spatial Visualization:
   Implement interactive maps using tools like Folium or Mapbox to visualize property locations, clusters, and price heatmaps more intuitively.
4. User Authentication & Dashboard:
   Develop personalized dashboards with login functionality, enabling users to save search preferences and view tailored recommendations.
5. Sentiment Analysis of Reviews:
   Extend the system by analyzing user or agent reviews to assess the sentiment and reliability of property listings.
6. Mobile App Deployment:
   Transform the Streamlit web platform into a mobile-friendly application to enhance accessibility and usability for end-users.

Appendix:

Section 11: Perform Measurement of complexity with Halstead Metrics for chosen system.

Modules Included:

1. **Data Scraper**
   Scrapes real estate data (prices, locations, area, etc.) from the 99acres website using tools like Beautiful Soup/Selenium.
2. **Data Cleaner**
   Cleans and transforms raw scraped data (e.g., handling nulls, parsing area, converting strings to numbers).
3. **EDA (Exploratory Data Analysis)**
   Uses pandas, seaborn, and matplotlib to visualize distributions, correlations, trends in pricing, and locality data.
4. **Feature Engineering**
   Constructs new meaningful features (like price per sqft, location encoding, age of property).
5. **Model Trainer**
   Trains machine learning models (e.g., Linear Regression, Random Forest) to predict property prices.
6. **Hyperparameter Tuning Module**
   Uses Optuna to tune hyperparameters for optimal model performance.
7. **Deployment Module (Streamlit)**
   Hosts interactive dashboard where users can input property features and get price predictions + visual insights.

Assumptions Across Modules:

Based on the complexity and functionalities in the above modules:

- **Unique Operators (n_1):** 145
  Includes loops, conditionals, preprocessing, ML pipeline operations, API calls, visualizations, etc.
- **Unique Operands (n_2):** 210
  Variables like price_data, area_sqft, location_df, model_score, optuna_trial, etc.
- **Total Operators (N_1):** 890
  Function calls, pandas/NumPy operations, model.fit(), seaborn plots, etc.
- **Total Operands (N_2):** 980
  Column names, intermediate results, training data, encoded values, prediction outputs.

Metrics Calculation:

1. **Vocabulary (n):**
   $n = n_1 + n_2$
   $n = 145 + 210 = 355$

2. **Program Length (N):**
   $N = N_1 + N_2$
   $N = 890 + 980 = 1870$

3. **Volume (V):**
   $V = N \times \log_2(n)$
   $V = 1870 \times \log_2(355)$
   $V \approx 1870 \times 8.47 \approx 15838.9$

4. **Difficulty (D):**
   $D = (n_1 / 2) \times (N_2 / n_2)$
   $D = (145 / 2) \times (980 / 210)$
   $D \approx 72.5 \times 4.67 \approx 338.58$

5. **Effort (E):**
   $E = D \times V$
   $E= 338.58 \times 15838.9 \approx 5360693.54$

<u>Summary of Results:</u>

Metric	**Value**
Vocabulary (n)	355
Program Length (N)	1870
Volume (V)	~15838.9
Difficulty (D)	338.58
Effort (E)	~5360693.54